Integrating the µOS++ Test Harness for AArch32 / Raspberry Pi Zero 2 W

> **Earlier layout — read as history.** This document describes an earlier harness layout. The SMP kernel and test harness are now unified in `micro-os-plus-iii` (branch `smp`) alongside the sibling ports `micro-os-plus-iii-{posix-arch,cortexm,aarch32,aarch64}`. Since 2026-10-06 there is **no `micro-os-plus-iii-devices`**: its drivers and SoC support are part of each port (`drivers/`, `soc/<chip>/`), so skip every clone, symlink or `UOS_DEVICES_DIR` for it below. Everything runs from `micro-os-plus-iii/tests`: `xpm run install-all`, then `xpm run test-all` (native + cortex + `test-smp-cmake`). For the current framework see `STEPS.md`; for every test, board and probe see `TESTS-CATALOG.md`; for how the `smp` branches were built, commit by commit, see `xpack-dev-smp.md` Parts II and III. The code is the truth where they disagree.
>
> How to take the test folder from `/home/dan/Work/micro-os-plus-iii/micro-os-plus-iii.git/tests` (the xpm/xPack harness) and make it test the **AArch32 port on the Raspberry Pi Zero 2 W**, reusing the SMP kernel, the devices repository and the `micro-os-plus-iii-aarch32` port.
>
> This document assumes the four sibling repositories `micro-os-plus-iii-smp`, `micro-os-plus-iii-devices`, `micro-os-plus-iii-aarch32` and `micro-os-plus-iii-aarch64` are available as working clones beside each other.
>
> **Status.** This document is the *design analysis* that led to the working setup. The verified, up-to-date, step-by-step procedure — C++20, the `micro-os-plus::platform` / `micro-os-plus::platform-support` split, the port's `test/boards/<board>/` layout, the strong semihosting, and the per-platform test/project names — is in **`WORK-SMP-AARCH32-AARCH64-HARNESS-GUIDE.md`**. Where an example below differs from that guide, the guide is authoritative.

---

1. Purpose and scope

You have two things that were not designed together:

1. **The xpack test harness** — the `tests/` folder under `micro-os-plus-iii.git`. It is a *matrix runner*: it declares `(platform × toolchain × build type)` as xpm build configurations and drives CMake/CTest, with one platform folder per target.
2. **The SMP/AArch32 project family** — a kernel (`-smp`), a driver/SiC package (`-devices`) and architecture ports (`-aarch32`, `-aarch64`). It has its *own* in-tree CMake test system: each board's applications live in the port's `test/<board>/` (with a `tests.cmake`), and the kernel's `test_smpl/` holds the two shared runners (`run-qemu.sh`, `run-hw.sh`).

The goal is to run the harness's portable suites — `rtos-apis`, `mutex-stress`, `cmsis-os-validator` — against the **AArch32 port on a Pi Zero 2 W**, under QEMU and on real silicon, in a structured and reproducible way.

This document explains both integration paths, tells you which one to pick and why, and gives concrete file skeletons.

---

2. The two systems, side by side

2.1 The xpack harness (the `test` folder you want to reuse)

Key traits (see the companion `TESTS-XPACK-SYSTEM.md` for the full analysis):

- Test **sources** are platform-agnostic: `tests/sources/<suite>/`, each an `INTERFACE` library with a `test::<suite>` alias.
- Test **platforms** are `tests/platforms/<name>/`, each with `cmake/definitions.cmake`, `cmake/dependencies-folders.cmake`, `cmake/platform-library.cmake` and a `CMakeLists.txt` that creates the executables and registers `add_test()`.
- The **matrix** lives in `tests/package.json → xpack.buildConfigurations`; each entry selects `platformName`, `toolchainFileName` and `buildType`.
- The build is a CMake **include chain**: `tests/CMakeLists.txt → tests-main.cmake →` platform fragments `→ add_subdirectory("..")` for the kernel `→ add_subdirectory(platforms/...)`.
- Tests obey one contract: `int os_main(int argc, char* argv[])` returns `0` on success; CTest checks the exit code.
- Runners are whatever command `add_test()` names: the host process, QEMU, or OpenOCD with Arm semihosting.

2.2 The SMP / AArch32 system

- `micro-os-plus-iii-smp` exports the **same alias** `micro-os-plus::iii` (a 37-source kernel core) plus optional groups (`iii-posix-io`, `iii-drivers`, `iii-semihosting`, ...) and `micro-os-plus::port-smp-decls`.
- `micro-os-plus-iii-devices` exports `micro-os-plus::devices` (SD, flatfs, DWC2, FatFs) and `micro-os-plus::soc-bcm2837`.
- `micro-os-plus-iii-aarch32` exports `micro-os-plus::aarch32`, which links `micro-os-plus::iii`, `micro-os-plus::port-smp-decls` and `micro-os-plus::soc-bcm2837`, and carries the board `-mcpu` flags as both compile and link options.
- Its test system uses `uos_add_app()` / `uos_add_test_app()` from `micro-os-plus-iii-smp/cmake/uos-app.cmake`; each board's applications live in the port's `test/<board>/` (with a `tests.cmake`); execution is via `micro-os-plus-iii-smp/test_smpl/run-qemu.sh` and `run-hw.sh`, with the per-port/board facts in the port's `test/boards/<board>/hw.sh`.
- Tests print a machine-greppable `RESULT: PASS|FAIL|SKIP` line and end with a semihosted `SYS_EXIT`; the runners parse the log.

2.3 Concept mapping

xpack harness concept	SMP / AArch32 equivalent	Notes
`tests/sources/<suite>` (`test::<suite>`)	`micro-os-plus-iii-smp/test_smpl/common/<app>` (`micro-os-plus::test-common`)	Both are ISA-neutral sources; both are built by the port.
`os_main()` exit code	`RESULT:` line + semihosted `SYS_EXIT`	The kernel entry is identical (`os_main`), the *verdict channel* differs.
`tests/platforms/<p>` (`micro-os-plus::platform`)	`test/boards/<board>` + `micro-os-plus::aarch32`	Both are the per-target glue.
`tests/package.json` build configurations	`cmake -DBOARD=...` + `test/hw.sh` env vars	xpm matrix vs. CMake option + env.
`tests/cmake/...` include chain	`micro-os-plus-iii-smp/cmake/uos-app.cmake`	Different helper sets, same idea.
`add_test()` → CTest	`run-qemu.sh` / `run-hw.sh`	CTest vs. bespoke runners.
		Equivalent.

xpack harness concept	SMP / AArch32 equivalent	Notes
Toolchain file `arm-none-eabi-gcc.cmake`	`micro-os-plus-iii-smp/cmake/toolchains/arm-none-eabi.cmake`	

The important compatibility fact: **the SMP kernel calls `os_main(argc, argv)` from `os-main.cpp`, exactly like the plain kernel.** The xpack test *bodies* are therefore reusable; only the console and the verdict channel need adapting.

---

3. Which integration should you choose?

There are two clean ways, and one trap.

	Strategy A — reuse the xpack harness	Strategy B — extend the AArch32 in-tree harness
What you add	A new `platforms/aarch32-rpi-zero-2w/` to the harness, plus build configs	The xpack suites as extra `test_smpl/common/` apps
Reuses	The harness structure, CTest, xpm matrix	The port, boot, MMU, SMP bring-up, hardware runner
Solves Pi boot/SMP/hardware for you	No — you must wire it	Yes — already done
Effort	High (top-level CMake surgery)	Low–medium (console adapter)
Best for	Reusing the *harness*, many platforms	Testing *this port* quickly and robustly

The trap: the xpack harness's `tests/cmake/tests-main.cmake` runs `add_subdirectory("..")` to build the **plain** kernel and define `micro-os-plus::iii`. If you also add the SMP kernel, CMake fails with a **duplicate alias** `micro-os-plus::iii`. You cannot have both kernels in one build. Strategy A therefore means pointing the harness at the SMP kernel instead of the plain one — i.e. it is not a drop-in.

Recommendation: use **Strategy B** to get tests running on the Zero 2 W today, and use **Strategy A′** (a *separate* harness project that consumes the SMP kernel and the AArch32 port) if you specifically want the xpm/CTest matrix and multi-platform structure.

Both are detailed below.

---

4. Strategy A — reuse the xpack `tests/` folder

4.1 Do not overlay the plain kernel

Because of the alias collision, the harness for AArch32 must be a **separate project** whose "library under test" is the **AArch32 port** — which itself brings the SMP kernel and the devices package. The cleanest layout:

Layout

```
workspace/
├── micro-os-plus-iii-smp/          # kernel + shared tests
├── micro-os-plus-iii-devices/      # drivers + bcm2837
├── micro-os-plus-iii-aarch32/      # the port
├── micro-os-plus-iii-aarch64/      # sibling port (optional)
└── aarch32-tests/                  # ← new harness project (a copy of tests/)
    ├── package.json                # xpm matrix
    ├── CMakeLists.txt              # entry point
    ├── cmake/                      # tests-main, common-options, global-definitions
    ├── sources/                    # rtos-apis, mutex-stress, cmsis-os-validator
```

```
    └── platforms/
        └── aarch32-rpi-zero-2w/
```

Copy the `tests/` folder to `aarch32-tests/` and change exactly one thing at the top of `cmake/tests-main.cmake`: replace the `add_subdirectory("..")` that builds the plain kernel with an `add_subdirectory` of the AArch32 port.

File: aarch32-tests/cmake/tests-main.cmake

```
# aarch32-tests/cmake/tests-main.cmake
# aarch32-tests/cmake/tests-main.cmake  (edited excerpt)
# Instead of add_subdirectory(".." "top-bin"):
add_subdirectory ("${UOS_AARCH32_DIR}" "port-bin")   # → iii + devices + aarch32
```

Resolve the directory as a sibling in `CMakeLists.txt` (the convention the AArch32 project already uses):

File: aarch32-tests/cmake/tests-main.cmake

```
# aarch32-tests/cmake/tests-main.cmake
get_filename_component (_sib "${CMAKE_CURRENT_SOURCE_DIR}/.." ABSOLUTE)
set (UOS_AARCH32_DIR "${_sib}/micro-os-plus-iii-aarch32" CACHE PATH "")
# The port finds the SMP kernel and the devices package as its own siblings;
# override only if they live elsewhere:
# set (UOS_SMP_DIR     "..." CACHE PATH "")
# set (UOS_DEVICES_DIR "..." CACHE PATH "")
```

Adding the SMP kernel or the devices package **separately** would make the port add them a second time and CMake would fail. Do **not** keep the harness's `add_subdirectory("..")` either; that is the plain kernel and would collide on `micro-os-plus::iii`.

4.2 The new platform directory

Create `platforms/aarch32-rpi-zero-2w/` with the standard four files.

`cmake/definitions.cmake`

File: aarch32-tests/platforms/aarch32-rpi-zero-2w/cmake/definitions.cmake

```
# aarch32-tests/platforms/aarch32-rpi-zero-2w/cmake/definitions.cmake
set (xpack_platform_compile_definition "MICRO_OS_PLUS_PLATFORM_AARCH32_RPI_ZERO_2W")
# The port supplies its own startup.S and vectors; there is no separate
# device xPack. xpack_device_* is therefore not used here.
```

`cmake/dependencies-folders.cmake` — the test sources plus the port.

File: aarch32-tests/platforms/aarch32-rpi-zero-2w/cmake/dependencies-folders.cmake

```
# aarch32-tests/platforms/aarch32-rpi-zero-2w/cmake/dependencies-folders.cmake
set (
  xpack_dependencies_folders
  "${CMAKE_SOURCE_DIR}/sources/mutex-stress"
  "${CMAKE_SOURCE_DIR}/sources/rtos-apis"
  "${CMAKE_SOURCE_DIR}/sources/cmsis-os-validator"
  # Already added by tests-main.cmake: the AArch32 port (which brings the
  # SMP kernel and the devices package). Do not list them again.
  # Portable xPacks, if a suite needs them:
  "${CMAKE_SOURCE_DIR}/xpacks/@xpacks/arm-cmsis-rtos-validator"
)
```

If the kernel/port were added by `tests-main.cmake` (as in §4.1), they must not be repeated here. If instead you keep `tests-main.cmake` untouched and let the *dependencies list* add them, then put them here and remove the `add_subdirectory("..")` line. Either way, exactly one place adds each.

`cmake/platform-library.cmake` — the per-target glue. This is where the board flags and the port link live.

File: [aarch32-tests/platforms/aarch32-rpi-zero-2w/cmake/platform-library.cmake]

```
# aarch32-tests/platforms/aarch32-rpi-zero-2w/cmake/platform-library.cmake
add_library (platform-aarch32-rpi-zero-2w-interface INTERFACE EXCLUDE_FROM_ALL)

target_include_directories (platform-aarch32-rpi-zero-2w-interface INTERFACE "include")

# The board's linker script comes from the AArch32 port repository.
set (_board_dir "${UOS_AARCH32_DIR}/test/boards/rpi-zero-2w")

target_compile_definitions (
  platform-aarch32-rpi-zero-2w-interface
  INTERFACE
    "${xpack_platform_compile_definition}"
    OS_NCPU=4
    OS_USE_SMP_SCHEDULER=1
    TRACE
    SEMIHOST
    __ARM_EABI__
    __ARM_ARCH_7A__
    SOC_BCM2837
    LED_PIN=29
)

set (
  _flags
  -mcpu=cortex-a53 -marm -mfloat-abi=hard -mfpu=neon-fp-armv8
  -Werror
)

target_compile_options (platform-aarch32-rpi-zero-2w-interface INTERFACE ${_flags})
target_link_options (
  platform-aarch32-rpi-zero-2w-interface
  INTERFACE ${_flags} -nostartfiles -Wl,--gc-sections
            "-T${_board_dir}/linker.ld"
)

# micro-os-plus::aarch32 pulls micro-os-plus::iii, port-smp-decls and soc-bcm2837.
target_link_libraries (
  platform-aarch32-rpi-zero-2w-interface
  INTERFACE micro-os-plus::aarch32
)

add_library (micro-os-plus::platform ALIAS platform-aarch32-rpi-zero-2w-interface)
```

Notes: - `OS_NCPU=4` and `OS_USE_SMP_SCHEDULER=1` are normally set by `uos_add_app()`, which the harness does not use, so set them here. - The `-mcpu`/`-mfpu` flags **must** appear at link time too, or the linker picks the wrong multilib and every AArch32 link fails with *"uses VFP register arguments"*. - Do not link `micro-os-plus::iii-posix-io` into this bare-metal build; it collides with newlib's `read`/`write` prototypes.

4.3 The platform `CMakeLists.txt` and the runners

The executable link set is the same as any other platform, except that `micro-os-plus::iii` and `micro-os-plus::platform` both resolve inside the SMP/port tree:

File: aarch32-tests/platforms/aarch32-rpi-zero-2w/CMakeLists.txt

```
# aarch32-tests/platforms/aarch32-rpi-zero-2w/CMakeLists.txt
include ("cmake/platform-library.cmake")

function (add_test_executable name)
  add_executable (${name})
  set_target_properties (${name} PROPERTIES OUTPUT_NAME "${name}")
  xpack_add_cross_custom_commands (${name})   # size + .bin
endfunction ()

if (ENABLE_MUTEX_STRESS_TEST)
  add_test_executable (mutex-stress-test)
  target_link_libraries (
    mutex-stress-test
    PRIVATE micro-os-plus::common-options
            test::mutex-stress
            micro-os-plus::iii
            micro-os-plus::platform
  )
  # QEMU, via the AArch64 boot shim (see below).
  add_test (
    NAME "${PLATFORM_NAME}-mutex-stress-test"
    COMMAND
      ${CMAKE_COMMAND} -E env
      UOS_QEMU_SHIM=${CMAKE_CURRENT_BINARY_DIR}/shim8.img
      bash "${CMAKE_CURRENT_SOURCE_DIR}/run-qemu-aarch32.sh"
      $<TARGET_FILE:mutex-stress-test>
  )
endif ()
```

The cross custom command must emit a raw `.bin`; the port's own build does this via `uos_add_app`. If `xpack_add_cross_custom_commands` does not, add an explicit `objcopy -O binary` POST_BUILD step (the Pi firmware and the shim both consume a raw image, not an ELF).

QEMU runner — the AArch32 boot shim

QEMU's `raspi3b` starts its Cortex-A53 cores in **AArch64**. The AArch32 image therefore needs the port's 20-line AArch64 stub that drops to AArch32 and jumps to the image. The AArch32 repository already builds it as `shim8.img` from `test/boards/rpi-zero-2w/qemu-raspi3-shim/`. A minimal wrapper:

Wrapper script (the AArch32 QEMU shim)

```
# wrapper script
#!/usr/bin/env bash
# run-qemu-aarch32.sh <image.bin>
set -euo pipefail
IMG="${1:?usage: run-qemu-aarch32.sh <image.bin>}"
SHIM="${UOS_QEMU_SHIM:?set UOS_QEMU_SHIM to shim8.img}"
QEMU="$(ls "$HOME"/.local/xPacks/@xpack-dev-tools/qemu-arm/*/.content/bin/qemu-system-aarch64 |
        tail -1)"
exec "$QEMU" -M raspi3b -smp 4 -nographic -serial none \
  -semihosting-config enable=on,target=native \
  -kernel "$SHIM" -device "loader,file=${IMG},addr=0x10000"
```

A semihosted `SYS_EXIT` (which `std::exit()` produces on a `SEMIHOST` build) terminates QEMU with the test's code, so **CTest gets a real pass/fail** with no log parsing. This is the one runner where the harness's exit-code model works unchanged.

> Do **not** run the `-mcpu=cortex-a53` image under `raspi2b`/Cortex-A7: the A53 build emits ARMv8 `lda`/`stl` atomics that an A7 cannot execute. Use `raspi3b` + shim.

Hardware runner — semihosting via the `aarch64` target

On real silicon the Pi's OpenOCD config uses an **`aarch64` debug target with a CTI** (the ARMv7 `cortex_a` target cannot halt a Cortex-A53). That target does **not** intercept the classic AArch32 semihosting trap (`SVC 0x123456`), but it **does** support the `HLT #0xF000` form — which is exactly what the port's `semihosting.hpp` emits (`0xE10F0070`). The session therefore works much like the Cortex-M ones, with three differences:

- `arm semihosting enable` is issued **per core** (`run-hw.sh` does this for every core it drives); a core with semihosting off stalls at the first debug halt.
- The verdict is read from the **semihosted console** that OpenOCD prints into its own log — the same `RESULT: PASS|FAIL|SKIP` protocol as QEMU. Under `SEMIHOST` the PL011 UART is mirrored byte-for-byte to that channel, so the operator can also watch it on their own terminal.
- There is no GDB and no reset; the session is pure OpenOCD (halt, load, start core 0 with the CPSR mode forced, resume cores 1–3 where the firmware parked them — the kernel releases them through mailbox 3).

A CTest wrapper can simply run the port's session and map the log to an exit code:

CTest wrapper script

```
#!/usr/bin/env bash
# run-hw-aarch32.sh <app-name> [run-seconds]
exec "${UOS_AARCH32_DIR}/test/hw.sh" "$1" "${2:-}"
```

If you drive OpenOCD directly instead, grep the log for the `RESULT:` line (and treat a lost debug link as an error, not a test failure):

OpenOCD command

```
timeout 300 openocd -f "$UOS_HW_CFG" -c init -c halt \
  -c "load_image ${IMG}" -c "arm semihosting enable" -c resume >"$LOG" 2>&1 || true
grep -q 'RESULT: PASS' "$LOG" && exit 0
grep -q 'RESULT: SKIP' "$LOG" && exit 77      # CTest SKIP_RETURN_CODE
exit 1
```

Prefer wrapping the port's runner: it already starts core 0 alone with the CPSR mode forced (cores 1–3 stay in the firmware's loop until the kernel releases them), the one-test-per-power-cycle rule and DEBUG LINK LOST detection.

Because hardware is one-test-per-power-cycle, mark these CTest tests `RUN_SERIAL` and do not include them in an unattended suite.

4.4 `package.json` build configurations

Add a pair (debug/release) for the QEMU run, and optionally for hardware:

File: [aarch32-tests/package.json]

```
// aarch32-tests/package.json
"aarch32-rpi-zero-2w-cmake-gcc-debug": {
  "inherit": [
    "cmake-actions",
    "aarch32-dependencies"          // a new hidden block, below
  ],
  "properties": {
    "buildType": "Debug",
```

```
    "platformName": "aarch32-rpi-zero-2w",
    "toolchainFileName": "arm-none-eabi-gcc.cmake"
  }
},
"aarch32-rpi-zero-2w-cmake-gcc-release": {
  "inherit": ["aarch32-rpi-zero-2w-cmake-gcc-debug"],
  "properties": { "buildType": "MinSizeRel" }
}
```

The hidden dependency block pins the toolchain and, if the harness copies the repos rather than using siblings, the QEMU/AArch64 tools:

File: `aarch32-tests/package.json`

```
// aarch32-tests/package.json
"aarch32-dependencies": {
  "hidden": true,
  "devDependencies": {
    "@xpack-dev-tools/arm-none-eabi-gcc": "15.2.1-1.1.1",
    "@xpack-dev-tools/aarch64-none-elf-gcc": "15.2.1-1.1.1",
    "@xpack-dev-tools/qemu-arm": "9.2.4-1.1"
  }
}
```

Wire the new configuration into an aggregate action so it participates in `test-all`:

File: `aarch32-tests/package.json`

```
// aarch32-tests/package.json
"test-aarch32-rpi-zero-2w-cmake": [
  "xpm run prepare --config aarch32-rpi-zero-2w-cmake-gcc-debug",
  "xpm run build   --config aarch32-rpi-zero-2w-cmake-gcc-debug",
  "xpm run test    --config aarch32-rpi-zero-2w-cmake-gcc-debug",
  "xpm run prepare --config aarch32-rpi-zero-2w-cmake-gcc-release",
  "xpm run build   --config aarch32-rpi-zero-2w-cmake-gcc-release",
  "xpm run test    --config aarch32-rpi-zero-2w-cmake-gcc-release"
]
```

4.5 Test-source adaptations for the bare-metal port

The harness's suites are written for the plain, hosted kernel. Against the AArch32 port they need three adjustments.

1. **Console.** The harness tests call `printf`/`puts`/`trace::printf`. The port's console is `uart.hpp` (and, under `SEMIHOST`, the semihosting channel). Provide a small shim so `printf` reaches the port's UART, or include the port's `uart.hpp` and replace the calls. `mutex-stress` is the simplest to port because it prints little.

2. **Verdict channel.** CTest wants an exit code; the port's runner wants a `RESULT:` line. Do both: keep `os_main` returning `0`/non-zero *and* print `RESULT: PASS` / `RESULT: FAIL` before returning. That makes the same binary usable by CTest (QEMU semihosting exit) and by `run-hw.sh` (semihosted console in OpenOCD's log). A two-line helper is enough.

3. **Optional layers.** `rtos-apis` links `xpacks::chan-fatfs` and exercises POSIX I/O; the AArch32 bare-metal build compiles neither `iii-posix-io` nor a FatFS port for the Pi. Build the SoC-neutral subset first: `mutex-stress` (pure RTOS), then `cmsis-os-validator` (validator + one IRQ), and only then decide whether `rtos-apis` should be trimmed or given a FatFS backend from `micro-os-plus::devices`.

`cmsis-os-validator` also needs the `@xpacks/arm-cmsis-rtos-validator` source xPack on the include path and an interrupt source. The Pi's `timer_arm.hpp`/mailbox IPI can supply it; the CMSIS-OS compatibility layer is in the SMP kernel.

4. **`os-app-config.h`.** Every suite ships its own `include/cmsis-plus/os-app-config.h`. The AArch32 port supplies the port contract in `include/cmsis-plus/rtos/port/os-c-decls.h` (which sets `OS_NCPU`, `OS_USE_SMP_SCHEDULER`, the SMP lock types, …). Keep the suite's app config minimal and let the port header own the port facts, exactly as the port's own `include/cmsis-plus/os-app-config.h` does.

4.6 Toolchain

Use the harness's `arm-none-eabi-gcc.cmake` toolchain file (build-helper), or the port's own `micro-os-plus-iii-smp/cmake/toolchains/arm-none-eabi.cmake`. Both set `CMAKE_SYSTEM_NAME Generic` and the `arm-none-eabi-*` tools. The port is compiled with `arm-none-eabi-gcc` 15; the AArch64 shim with `aarch64-none-elf-gcc` 15.

4.7 Running

Commands

```
# once
npm --prefix aarch32-tests install
xpm run install-aarch32-rpi-zero-2w-cmake -C aarch32-tests   # if defined

# QEMU (A53 + shim), debug + release
xpm run test-aarch32-rpi-zero-2w-cmake -C aarch32-tests

# one step at a time
xpm run prepare --config aarch32-rpi-zero-2w-cmake-gcc-debug -C aarch32-tests
xpm run build   --config aarch32-rpi-zero-2w-cmake-gcc-debug -C aarch32-tests
xpm run test    --config aarch32-rpi-zero-2w-cmake-gcc-debug -C aarch32-tests
```

---

5. Strategy B — extend the AArch32 in-tree harness (recommended)

Here you keep the port's CMake project and runner, and add the harness's suites as extra shared applications. This is far less work because boot, MMU, SMP bring-up and the hardware session are already solved.

5.1 Where the sources go

Put each suite in the shared test tree so every port can build it:

Layout

```
micro-os-plus-iii-smp/test_smpl/common/
├── mutex-stress/          # copied from the harness, adapted
│   ├── main.cpp
│   └── ...
├── rtos-apis/
└── cmsis-os-validator/
```

and register them in the board's `test/<board>/tests.cmake`:

File: [micro-os-plus-iii-aarch32/test/rpi-zero-2w/tests.cmake]

```
# micro-os-plus-iii-aarch32/test/rpi-zero-2w/tests.cmake
set (UOS_TEST_APPS
     sd_test smp_test0 smp_test1 smp_test2 smp_test3 smp_test4
     smp-mat-test smp-mat-sdcard-test smp-num-test
```

```
    smp-pipeline-test smp-pro-cons-test usb_test
    mutex-stress            # ← added
    CACHE INTERNAL "Shared µOS++ III SMP test applications")
```

Then `uos_add_test_app` finds them by name, because it globs `test_smpl/common/<app>/*.cpp`.

5.2 Console and verdict adapter

The SMP tests use `test_smpl/common/include/test-console.hpp` (`console()`, `console_uart()`, `report_per_core()`) and stop the run through `hw_result.hpp`. Adapt the harness tests to the same helpers:

Test source (`os_main` adapter)

```
#include <test-console.hpp>     // console(), console_uart() — serialised, multi-core safe
#include <hw_result.hpp>        // hw_result::ok() / hw_result::fail()

int
os_main (int argc, char* argv[])
{
  int rc = run_the_test ();

  if (rc == 0)
    {
      console ("RESULT: PASS\n");
      hw_result::ok ();       // semihosted SYS_EXIT success; no-op off SEMIHOST
    }
  else
    {
      console ("RESULT: FAIL\n");
      hw_result::fail ();     // semihosted SYS_EXIT failure; no-op off SEMIHOST
    }

  return rc;
}
```

If you prefer to keep the harness sources untouched, add a tiny adapter `test_smpl/common/include/stdio-shim.hpp` that maps `printf`/`puts` onto `test::console()` and include it from a wrapper translation unit. Either way, the verdict must reach the UART so `run-hw.sh` can grep it.

5.3 Build it

Nothing else is needed: the AArch32 `test/CMakeLists.txt` loops over `UOS_TEST_APPS` and builds each for `{qemu, hwd}`. Adding the suite to the list is sufficient.

Commands

```
cd micro-os-plus-iii-aarch32
cmake -S . -B build \
      -DCMAKE_TOOLCHAIN_FILE=../micro-os-plus-iii-smp/cmake/toolchains/arm-none-eabi.cmake
cmake --build build -j8
ls build/test/mutex-stress-*        # mutex-stress-qemu, mutex-stress-hwd, .bin, .map
```

5.4 Run it

QEMU (A53 + shim, the port's supported path):

Commands

```
UOS_QEMU_SHIM=build/test/shim8.img UOS_QEMU_LOAD_ADDR=0x10000 \
../micro-os-plus-iii-smp/test_smpl/run-qemu.sh build/test \
    "$(ls ~/.local/xPacks/@xpack-dev-tools/qemu-arm/*/.content/bin/qemu-system-aarch64 | tail
        -1)" \
    -M raspi3b -smp 4
```

Hardware (J-Link, one test per power cycle):

Commands

```
cd micro-os-plus-iii-aarch32
test/hw.sh list
test/hw.sh mutex-stress
```

---

6. Raspberry Pi Zero 2 W specifics

These facts are what make the Pi different from the Cortex-M boards the harness already supports.

Topic	Fact
Silicon	BCM2837/BCM2710A1, 4× Cortex-A53, run in **AArch32** (ARMv7-A).
Boot	`config.txt`: `arm_64bit=0`, `kernel=kernel7.img`, `kernel_address=0x010000`, `enable_uart=1`, `gpu_mem=16`. The GPU firmware starts the cores in 32-bit mode.
Link	`test/boards/rpi-zero-2w/linker.ld`: `ORIGIN = 0x10000`, so **one binary** runs under QEMU (`-kernel` loads at 0x10000) and on hardware (`kernel_address`).
QEMU	`raspi3b` + the AArch64→AArch32 `shim8.img`; the image is loaded at `0x10000` with `-device loader`. **Not** `raspi2b`.
Console	PL011 UART0 on GPIO14/15, 115200 8N1; under `SEMIHOST` also the semihosting channel.
Hardware debug	OpenOCD with an **`aarch64` target + CTI** (ARMv7 `cortex_a` cannot halt an A53). Configs: `openocd-jlink-rpi3.cfg` (J-Link), `openocd-olimex.cfg` (Olimex) or `openocd-cmsisdap-rpi3.cfg` (CMSIS-DAP), all sourcing `rpi3-aarch32.cfg` → `bcm2837-aarch32.cfg`.
Semihosting on hardware	Uses the `HLT #0xF000` form (not the classic `SVC 0x123456` trap), which the OpenOCD `aarch64` target supports; `arm semihosting enable` is issued per core and the verdict is read from OpenOCD's log.
Reset	The Pi exposes only TRST, no SRST; the A53 debug target has no reset method. The runner does a deterministic halt/load/resume and **one test per power cycle**.
SMP	`OS_NCPU=4`, `OS_USE_SMP_SCHEDULER=1`; secondary cores park in `startup.S` and are released via `__smp_spin` + `SEV`.
LED	Onboard green ACT LED is **GPIO29**; `led.hpp`'s fallback (GPIO16) blinks nothing on a bare board.
Timers	ARM generic timer (1 ms tick) + mailbox IPI for cross-core reschedule (no GIC/SGIs).

---

7. Toolchains and reproducibility

Tool	Used for	Where
`arm-none-eabi-gcc` 15	AArch32 build	xPack, `~/.local/xPacks/@xpack-dev-tools/`
`aarch64-none-elf-gcc` 15	AArch64 port and the AArch32 QEMU shim	xPack
`qemu-system-aarch64`	both QEMU suites	xPack `@xpack-dev-tools/qemu-arm` 9.x
`openocd`	hardware sessions	xPack (or the distro's)
CMake ≥ 3.20	all builds	xPack or system

Pin exact versions when it matters:

Commands

```
cmake -DUOS_TOOLCHAIN_BIN=~/.local/xPacks/@xpack-dev-tools/arm-none-eabi-gcc/
        15.2.1-1.1.1/.content/bin ...
```

In the xpack harness, pin the same versions in a hidden `aarch32-dependencies` block so the matrix is reproducible.

---

8. Extending coverage

Once one suite runs on the Zero 2 W, the same pattern covers:

- **AArch64** — same silicon, 64-bit; the `-aarch64` port already has the same `test/CMakeLists.txt` shape. In the harness, add a sibling platform with `-mcpu=cortex-a53` (no `-marm`), the AArch64 toolchain and a direct `-kernel` QEMU run (no shim).
- **More boards** — the AArch32 port already parameterises `zero2w`, `rpi3b` and `luckfox-lyra` via `-DBOARD=`. Each becomes a platform in the harness.
- **More suites** — add `sources/<suite>/` + a platform block, as in the companion document; the CTest `add_test()` registration makes it part of every aggregate automatically.
- **Single-CPU ports** — `UOS_TEST_APPS_SMP_ONLY` already lets a port skip the SMP tests while building the rest.

---

9. Caveats and gotchas

- **Alias collision.** Never add both the plain kernel and the SMP kernel to one CMake build; both export `micro-os-plus::iii`.
- **`raspi2b` vs `raspi3b`.** The `cortex-a53` build must run on `raspi3b` + shim; a `cortex-a7` build may run on `raspi2b` directly. Mixing them faults on the first `lda`/`stl`.
- **Machine flags at link time.** Omit `-mcpu`/`-mfpu` from the link and the link fails with *"uses VFP register arguments"*.
- **AArch32 semihosting on hardware uses `HLT #0xF000`, not `SVC 0x123456`.** The classic trap is not intercepted by the `aarch64` OpenOCD target; enable semihosting per core and read the verdict from OpenOCD's log.
- **One test per power cycle** on hardware; mark hardware tests `RUN_SERIAL` and keep them out of unattended suites.

- **Rebuild before you test.** A stale AArch32 image is indistinguishable from a firmware bug. The per-test `test-*` actions now reconfigure + build before they run `ctest`, so that is handled for you; the bare `test` action still runs `ctest` only.
- **`-Werror` and the port's `-Wno-*`.** The port already suppresses the few warnings it needs; a copied suite may need its own local `#pragma` push/pop.
- **Two QEMU suites at once starve a vCPU** and look like a deadlock; run one suite on an idle host.
- **`rtos-apis` is not portable as-is** (POSIX I/O + FatFS); start with `mutex-stress`.
- **Console re-entrancy.** The SMP tests serialise output with a mutex; a multi-core suite that prints must do the same, or `RESULT:` lines can be split and the runner's grep will miss them.

---

10. Swapping the kernel in the Work folder (`micro-os-plus-iii` → `-smp`)

This chapter is the concrete recipe for taking the working folder

Layout

```
~/Work/micro-os-plus-iii/micro-os-plus-iii.git/     # plain kernel + tests/
```

and replacing the plain kernel with the SMP family, then adapting the harness's `tests/` folder so it builds and runs against the AArch32 port on the Zero 2 W.

10.1 The one rule that decides the layout

Only one project may define `micro-os-plus::iii`. The AArch32 port already adds the SMP kernel and the devices repository *itself*:

File: [micro-os-plus-iii-aarch32/CMakeLists.txt]

```
# micro-os-plus-iii-aarch32/CMakeLists.txt
# micro-os-plus-iii-aarch32/CMakeLists.txt
add_subdirectory ("${UOS_SMP_DIR}"     micro-os-plus-iii-smp)   # micro-os-plus::iii
add_subdirectory ("${UOS_DEVICES_DIR}" micro-os-plus-iii-devices)
add_library (micro-os-plus::aarch32 ...)
```

So the AArch32 port must be the **single entry point**, and the harness's `tests/` folder must sit where `add_subdirectory("..")` resolves to that port. Do **not** add the SMP kernel and the port in the same build — that is the duplicate-alias trap.

10.2 Prepare the Work folder

Keep the old tree for diffing against upstream, clone the SMP family beside it, and copy the harness into the AArch32 project:

Commands

```
cd ~/Work/micro-os-plus-iii

# Clone the SMP family from the bare repositories you already have.
git clone /home/dan/Downloads/GIT/micro-os-plus-iii-smp.git     micro-os-plus-iii-smp
git clone /home/dan/Downloads/GIT/micro-os-plus-iii-devices.git micro-os-plus-iii-devices
git clone /home/dan/Downloads/GIT/micro-os-plus-iii-aarch32.git micro-os-plus-iii-aarch32
git clone /home/dan/Downloads/GIT/micro-os-plus-iii-aarch64.git micro-os-plus-iii-aarch64   #
        optional

# Bring the harness across. Note: tests/ (plural), not the port's test/ (singular).
cp -a micro-os-plus-iii.git/tests micro-os-plus-iii-aarch32/tests
```

Resulting layout:

Layout

```
~/Work/micro-os-plus-iii/
├── micro-os-plus-iii.git/          # plain kernel — retired from the test build
│   └── tests/                      # (keep intact for the Cortex-M/native work)
├── micro-os-plus-iii-smp/          # SMP kernel  (micro-os-plus::iii)
├── micro-os-plus-iii-devices/      # drivers + BCM2837
├── micro-os-plus-iii-aarch32/      # ← the project that now owns the harness
│   ├── test/                       # the port's own in-tree suite (singular)
│   └── tests/                      # the xpack harness (plural), adapted below
└── micro-os-plus-iii-aarch64/      # optional sibling
```

The two test trees coexist because the port's own `test/` is added only when the port is the top-level project (`if (PROJECT_IS_TOP_LEVEL ...)`). Driving the build from `tests/` makes the port a subdirectory, so its `test/` is skipped and only the harness runs. Keep the names distinct (`test` vs `tests`) anyway.

10.3 Why the swap works with no edit to `tests-main.cmake`

`tests/cmake/tests-main.cmake` contains exactly one line that binds the harness to "the library under test":

File: [micro-os-plus-iii-aarch32/tests/cmake/tests-main.cmake]

```
# micro-os-plus-iii-aarch32/tests/cmake/tests-main.cmake
add_subdirectory ("..." "top-bin")
```

With `tests/` inside the AArch32 repository, `..` **is** the AArch32 project, so that single line now adds the port, which in turn adds the SMP kernel and the devices package. No edit is needed, and there is still exactly one `micro-os-plus::iii`.

Layout

```
tests/CMakeLists.txt
└── include cmake/tests-main.cmake
    ├── include cmake/common-options.cmake
    ├── include platforms/${PLATFORM_NAME}/cmake/{definitions,dependencies-folders}.cmake
    ├── xpack_add_dependencies_subdirectories(...)   # test sources + xpacks
    ├── add_subdirectory("..." "top-bin")            # → AArch32 port → SMP + devices
    └── add_subdirectory("platforms/${PLATFORM_NAME}" "platform-bin")  # exes + add_test
```

If the SMP and devices trees are **not** siblings of the AArch32 repo, point at them explicitly at configure time:

Commands

```
cmake ... -DUOS_SMP_DIR=/abs/path/micro-os-plus-iii-smp \
          -DUOS_DEVICES_DIR=/abs/path/micro-os-plus-iii-devices
```

10.4 If you must keep `tests/` under the SMP repository

The literal reading — "the tests folder's parent becomes `micro-os-plus-iii-smp`" — cannot work by itself, because the SMP kernel has no port and does not compile alone. If you really want that layout, insert a thin **wrapper** project and put the harness under it, leaving the SMP repo merge-clean:

Layout

```
micro-os-plus-iii-aarch32-harness/
├── CMakeLists.txt          # adds the port (which adds SMP + devices)
```

```
├── package.json
└── tests/                  # the harness
```

File: micro-os-plus-iii-aarch32-harness/CMakeLists.txt

```
# micro-os-plus-iii-aarch32-harness/CMakeLists.txt
# micro-os-plus-iii-aarch32-harness/CMakeLists.txt
cmake_minimum_required (VERSION 3.20)
get_filename_component (_sib "${CMAKE_CURRENT_SOURCE_DIR}/.." ABSOLUTE)
set (UOS_AARCH32_DIR "${_sib}/micro-os-plus-iii-aarch32" CACHE PATH "")
add_subdirectory ("${UOS_AARCH32_DIR}" "aarch32-bin")   # brings iii + devices + port
```

and change `tests-main.cmake`'s single `add_subdirectory("..")` to `add_subdirectory("${UOS_AARCH32_DIR}" "top-bin")` (or keep `..` if `tests/` sits directly under this wrapper and the wrapper adds the port — then `..` is the wrapper, which is fine). This keeps the SMP repo untouched. The rest of this chapter applies unchanged.

10.5 Adapt `tests/package.json`

1. **Add the AArch32 build configurations** (debug + release) exactly as in §4.4, with `platformName: "aarch32-rpi-zero-2w"` and `toolchainFileName: "arm-none-eabi-gcc.cmake"`.
2. **Pin the SMP-era tools** in a hidden block: `@xpack-dev-tools/arm-none-eabi-gcc` 15.x, `@xpack-dev-tools/aarch64-none-elf-gcc` 15.x (for the QEMU shim), `@xpack-dev-tools/qemu-arm` 9.x.
3. **Retire the plain-kernel platforms.** `nucleo-*` and `raspberrypi-pico` still link `micro-os-plus::iii-cortexm`, which the SMP kernel does not provide. (The `qemu-cortex-m0/m3/m4f/m7f` platforms were instead reworked to link the local Cortex-M port's generic cores, and `native` to the POSIX-arch port.) Remove their entries from `buildConfigurations`, from every `dependencies-folders.cmake` you keep, and from the `install-ci` / `test-ci` / `test-all` aggregates. Leave the old harness under `micro-os-plus-iii.git` for that work.

10.6 Adapt `tests/CMakeLists.txt`

Two standards change because the port is built as bare metal with the SMP kernel's expectations:

File: micro-os-plus-iii-aarch32-harness/tests/CMakeLists.txt

```
# micro-os-plus-iii-aarch32-harness/tests/CMakeLists.txt
# tests/CMakeLists.txt
set (CMAKE_C_STANDARD 11)
set (CMAKE_C_STANDARD_REQUIRED ON)
# set (CMAKE_C_EXTENSIONS OFF)     # the port compiles C as gnu11

set (CMAKE_CXX_STANDARD 20)
set (CMAKE_CXX_STANDARD_REQUIRED ON)
set (CMAKE_CXX_EXTENSIONS OFF)
```

Leave the `CMAKE_MODULE_PATH`/build-helper include and `enable_testing()` untouched; they are already correct.

10.7 The platform directory

Create `tests/platforms/aarch32-rpi-zero-2w/`. The port is already added by `add_subdirectory("..")`, so `dependencies-folders.cmake` lists only the test sources and the portable xPacks — **not** the port, kernel or devices:

File: micro-os-plus-iii-aarch32-harness/tests/platforms/aarch32-rpi-zero-2w/cmake/dependencies-folders.cmake

```
# micro-os-plus-iii-aarch32-harness/tests/platforms/aarch32-rpi-zero-2w/cmake/dependencies-
        folders.cmake
# tests/platforms/aarch32-rpi-zero-2w/cmake/dependencies-folders.cmake
set (
  xpack_dependencies_folders
  "${CMAKE_SOURCE_DIR}/sources/mutex-stress"
  "${CMAKE_SOURCE_DIR}/sources/rtos-apis"
  "${CMAKE_SOURCE_DIR}/sources/cmsis-os-validator"
  "${CMAKE_SOURCE_DIR}/xpacks/@xpacks/arm-cmsis-rtos-validator"
)
```

`definitions.cmake` just names the platform:

File: [micro-os-plus-iii-aarch32-harness/tests/platforms/aarch32-rpi-zero-2w/cmake/definitions.cmake]

```
# micro-os-plus-iii-aarch32-harness/tests/platforms/aarch32-rpi-zero-2w/cmake/definitions.cmake
set (xpack_platform_compile_definition "MICRO_OS_PLUS_PLATFORM_AARCH32_RPI_ZERO_2W")
```

`platform-library.cmake` links the port and adds what `uos_add_app()` would otherwise have added (bare-metal flags, the linker script, `OS_NCPU`):

File: [micro-os-plus-iii-aarch32-harness/tests/platforms/aarch32-rpi-zero-2w/cmake/platform-library.cmake]

```
# micro-os-plus-iii-aarch32-harness/tests/platforms/aarch32-rpi-zero-2w/cmake/platform-
        library.cmake
add_library (platform-aarch32-rpi-zero-2w-interface INTERFACE EXCLUDE_FROM_ALL)
target_include_directories (platform-aarch32-rpi-zero-2w-interface INTERFACE "include")

set (_board_dir "${UOS_AARCH32_DIR}/test/boards/rpi-zero-2w")

target_compile_definitions (
  platform-aarch32-rpi-zero-2w-interface
  INTERFACE
    "${xpack_platform_compile_definition}"
    OS_NCPU=4
    OS_USE_SMP_SCHEDULER=1
    TRACE SEMIHOST __ARM_EABI__ __ARM_ARCH_7A__ SOC_BCM2837
    LED_PIN=29
)

# The port's interface already carries -mcpu=cortex-a53 -marm -mfloat-abi=hard
# -mfpu=neon-fp-armv8 as both compile and link options. Add only the bare-metal
# extras uos_add_app() normally supplies.
target_compile_options (
  platform-aarch32-rpi-zero-2w-interface
  INTERFACE -fno-exceptions -fno-rtti -fabi-version=0
)
target_link_options (
  platform-aarch32-rpi-zero-2w-interface
  INTERFACE -nostartfiles -Wl,--gc-sections
            "-T${_board_dir}/linker.ld"
)

target_link_libraries (
  platform-aarch32-rpi-zero-2w-interface
  INTERFACE micro-os-plus::aarch32      # pulls iii + port-smp-decls + soc-bcm2837
)

add_library (micro-os-plus::platform ALIAS platform-aarch32-rpi-zero-2w-interface)
```

`CMakeLists.txt` creates the executables and registers the runners exactly as in §4.3, with one addition: because the harness does not use `uos_add_app()`, add an explicit raw-image step for the QEMU shim:

File: micro-os-plus-iii-aarch32-harness/tests/platforms/aarch32-rpi-zero-2w/CMakeLists.txt

```
# micro-os-plus-iii-aarch32-harness/tests/platforms/aarch32-rpi-zero-2w/CMakeLists.txt
add_custom_command (
  TARGET ${name} POST_BUILD
  COMMAND ${CMAKE_OBJCOPY} -O binary "$<TARGET_FILE:${name}>" "$<TARGET_FILE:${name}>.bin"
  VERBATIM
)
```

and build `shim8.img` once (the port's own `test/CMakeLists.txt` is not active when the port is a subdirectory):

File: micro-os-plus-iii-aarch32-harness/tests/platforms/aarch32-rpi-zero-2w/CMakeLists.txt

```
# micro-os-plus-iii-aarch32-harness/tests/platforms/aarch32-rpi-zero-2w/CMakeLists.txt
set (_a64 "${UOS_AARCH32_DIR}/test/boards/rpi-zero-2w/qemu-raspi3-shim")
add_custom_command (
  OUTPUT "${CMAKE_CURRENT_BINARY_DIR}/shim8.img"
  COMMAND aarch64-none-elf-gcc -nostdlib "-Wl,-T,${_a64}/shim.ld" "${_a64}/shim.S"
          -o "${CMAKE_CURRENT_BINARY_DIR}/shim.elf"
  COMMAND aarch64-none-elf-objcopy -O binary
          "${CMAKE_CURRENT_BINARY_DIR}/shim.elf" "${CMAKE_CURRENT_BINARY_DIR}/shim8.img"
  DEPENDS "${_a64}/shim.S" "${_a64}/shim.ld" VERBATIM
)
add_custom_target (qemu-shim ALL DEPENDS "${CMAKE_CURRENT_BINARY_DIR}/shim8.img")
```

Do **not** add `-Werror` yet: the harness's aggressive warning set is stricter than the port's own, and the port sources already suppress the warnings they need. Add `-Werror` once the build is clean.

10.8 Adapt the test sources

The suite bodies are reusable because the SMP kernel calls `os_main(argc, argv)` too, but three things must change for a bare-metal, multi-core, UART-console port (same list as §4.5):

1. **Console** — route `printf`/`puts`/`trace::printf` to the port's `uart.hpp` (or `test-console.hpp`); the port compiles neither `iii-posix-io` nor a FatFS backend for the Pi.
2. **Verdict** — print `RESULT: PASS` / `RESULT: FAIL` *and* return the exit code, so the same binary works under QEMU semihosting (CTest) and under `run-hw.sh` (semihosted console in OpenOCD's log).
3. **App config** — keep the suite's `include/cmsis-plus/os-app-config.h` minimal; the port contract lives in `include/cmsis-plus/rtos/port/os-c-decls.h` (OS_NCPU, SMP scheduler, lock types).

Start with `mutex-stress` (pure RTOS, little output), then `cmsis-os-validator`, then decide whether `rtos-apis` should be trimmed or given a FatFS backend from `micro-os-plus::devices`.

10.9 Build and run

Commands

```
cd ~/Work/micro-os-plus-iii/micro-os-plus-iii-aarch32
npm --prefix tests install
xpm run install-aarch32-rpi-zero-2w-cmake -C tests      # if defined
xpm run test-aarch32-rpi-zero-2w-cmake    -C tests

# one configuration, step by step
xpm run prepare --config aarch32-rpi-zero-2w-cmake-gcc-debug -C tests
```

```
xpm run build   --config aarch32-rpi-zero-2w-cmake-gcc-debug -C tests
xpm run test    --config aarch32-rpi-zero-2w-cmake-gcc-debug -C tests
```

Hardware stays with the port's own runner, which is far more reliable than a CTest wrapper (it knows `__smp_spin`, the CPSR resume mode and the one-test-per-power-cycle rule):

Commands

```
cd ~/Work/micro-os-plus-iii/micro-os-plus-iii-aarch32
test/hw.sh list
test/hw.sh mutex-stress
```

10.10 Verification checklist

- [ ] `tests/` sits inside `micro-os-plus-iii-aarch32/` (so `..` is the port).
- [ ] `micro-os-plus-iii-smp` and `-devices` are siblings (or passed via `-D`).
- [ ] No platform or config still references `micro-os-plus::iii-cortexm` / `iii-posix-arch`. (Satisfied for `native`, `qemu-cortex-*` and the `cortexm-*`/`pico2*` platforms; `nucleo-*`/`raspberrypi-pico` remain.)
- [ ] `CMAKE_CXX_STANDARD` is 20.
- [ ] `platform-library.cmake` links `micro-os-plus::aarch32` and adds the linker script.
- [ ] QEMU runs on `raspi3b` + `shim8.img`, image loaded at `0x10000`.
- [ ] `os_main` prints `RESULT:` and returns the same verdict.
- [ ] A configure log shows exactly one `micro-os-plus::iii` alias.

10.11 Troubleshooting

Symptom	Cause	Fix
CMake error: alias target `micro-os-plus::iii` already exists	plain kernel and SMP kernel both added	ensure `tests/` is under the AArch32 repo; do not add the SMP repo separately
CMake error: source dir already used (add_subdirectory twice)	the port re-adds the SMP repo you also added	add only the port; let it add SMP/devices
`cannot find <cmsis-plus/rtos/port/os-decls.h>`	no port in the build	link `micro-os-plus::aarch32`; never add the SMP kernel alone
link error: *uses VFP register arguments*	`-mcpu`/`-mfpu` missing at link	link `micro-os-plus::aarch32` (carries link options)
compile errors mentioning C++ features	wrong standard	`set(CMAKE_CXX_STANDARD 20)`
QEMU faults at the first `lda`/`stl`	A53 image under `raspi2b`	use `raspi3b` + `shim8.img`
hardware run never prints `RESULT`	semihosting not enabled per core, or the `SVC 0x123456` form used instead of `HLT #0xF000`	enable `arm semihosting` per core; use `test/hw.sh`/`run-hw.sh`
`shim8.img` missing	the port's own `test/` is not active	add the shim custom command in the harness platform

11. Command cheat sheet

Commands

```
# --- Strategy B: build and run the port's own suite -------------------------
cd micro-os-plus-iii-aarch32
cmake -S . -B build -DCMAKE_TOOLCHAIN_FILE=../micro-os-plus-iii-smp/cmake/toolchains/arm-none-
        eabi.cmake
cmake --build build -j8

# QEMU (A53 + shim)
UOS_QEMU_SHIM=build/test/shim8.img UOS_QEMU_LOAD_ADDR=0x10000 \
../micro-os-plus-iii-smp/test_smpl/run-qemu.sh build/test \
  "$(ls ~/.local/xPacks/@xpack-dev-tools/qemu-arm/*/.content/bin/qemu-system-aarch64 | tail -1)"
        \
  -M raspi3b -smp 4

# hardware (J-Link), one test per power cycle
test/hw.sh list
test/hw.sh mutex-stress

# --- Strategy A: harness matrix ---------------------------------------------
npm --prefix aarch32-tests install
xpm run test-aarch32-rpi-zero-2w-cmake -C aarch32-tests

# --- Work-folder swap (kernel → SMP, harness → AArch32 project) --------------
cd ~/Work/micro-os-plus-iii
git clone /home/dan/Downloads/GIT/micro-os-plus-iii-smp.git     micro-os-plus-iii-smp
git clone /home/dan/Downloads/GIT/micro-os-plus-iii-devices.git micro-os-plus-iii-devices
git clone /home/dan/Downloads/GIT/micro-os-plus-iii-aarch32.git micro-os-plus-iii-aarch32
cp -a micro-os-plus-iii.git/tests micro-os-plus-iii-aarch32/tests
cd micro-os-plus-iii-aarch32
npm --prefix tests install
xpm run test-aarch32-rpi-zero-2w-cmake -C tests
```

---

12. File reference

Concern	Location
xpack harness	`micro-os-plus-iii.git/tests/`
Harness after the swap	`micro-os-plus-iii-aarch32/tests/` (see §10)
Harness analysis	`TESTS-XPACK-SYSTEM.md`
Kernel + shared runners	`micro-os-plus-iii-smp/` (`test_smpl/run-qemu.sh`, `test_smpl/run-hw.sh`)
Board tests	`<port>/test/<board>/` (applications + `tests.cmake`)
Shared-suite guide	`micro-os-plus-iii-smp/docs/test-smpl.md`
App helper	`micro-os-plus-iii-smp/cmake/uos-app.cmake`
Toolchains	`micro-os-plus-iii-smp/cmake/toolchains/`
QEMU runner	`micro-os-plus-iii-smp/test_smpl/run-qemu.sh`
Hardware runner	`micro-os-plus-iii-smp/test_smpl/run-hw.sh`
Drivers / BCM2837	`micro-os-plus-iii-aarch32/{drivers,soc/bcm2837}/` (formerly `micro-os-plus-iii-devices/`)
AArch32 port	`micro-os-plus-iii-aarch32/` (`test/boards/rpi-zero-2w/`, `test/hw.sh`)

Concern	Location
AArch32 board config	`test/boards/rpi-zero-2w/{config.txt,linker.ld,qemu-raspi3-shim/,openocd-*.cfg}`
Build guide (canonical)	`micro-os-plus-iii-smp/docs/building-aarch32-aarch64.md`

Companion to `TESTS-XPACK-SYSTEM.md`. Verified against the `master` branches of the `-smp`, `-devices`, `-aarch32` and `-aarch64` repositories and the `tests/` folder of `micro-os-plus-iii`.